

DEEP LEARNING

Lecture

Optimization Techniques
Momentum, RMSProp, Adam

Instructor: Zafar Iqbal

Agenda

- What is optimizer
- Simple updates
- Momentum
- RMSProp
- Adam
- Comparison

What is an Optimizer?

- An optimizer is a method that decides how a neural network should change its weights so that it makes fewer mistakes. **OR**
- An optimizer tells the model how to learn.
- When a model makes a prediction, it can be:
 - Correct
 - Wrong
- If it is wrong, the model needs to adjust itself so it performs better next time.
- The optimizer decides:
 - How much to change
 - Which direction to move
 - How fast to learn
- Without an optimizer, the model cannot improve.

Why Optimizers Matter

■ A good optimizer:

- Trains the model faster
- Helps reach better accuracy
- Avoids getting stuck or learning too slowly
- Makes training stable

■ A bad optimizer:

- Learns very slowly
- May never reach good performance
- Can make training unstable

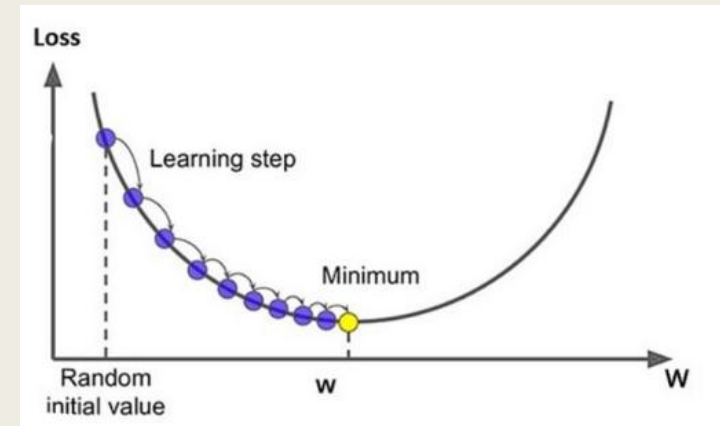
Example: Single Weight, One Step Learning

- Initial weight: $w = 10$
- Learning rate: $\alpha = 0.1$
- Gradient (feedback from loss):
- gradient = +4
 - $w = w - \text{learning_rate} * \text{gradient}$
 - $w = w - \alpha \times \text{gradient}$
 - $w = 10 - 0.1 \times 4$
 - $w = 10 - 0.4$
 - $w = 9.6$

■ What Happened?

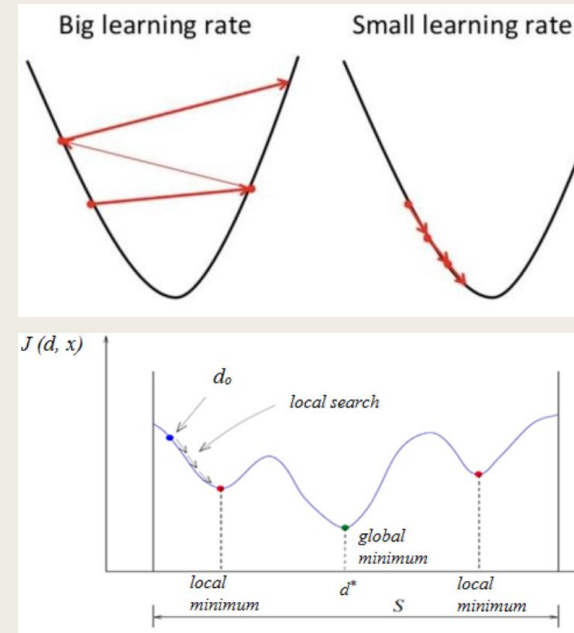
- Weight changed from **10** → **9.6**
- The model made a small correction
- Loss becomes smaller than before

- Optimizer decided **how much to correct**



Example: Another Step (Loss Keeps Reducing)

- Assume after updating:
 - New gradient becomes smaller (because loss reduced):
- gradient = +2
- Apply update again:
 - $w = 9.6 - 0.1 \times 2$
 - $w = 9.6 - 0.2$
 - $w = 9.4$
- **Observation:**
 - Gradient reduced: $4 \rightarrow 2$
 - Weight changes become smaller
 - Model is moving toward minimum loss
- This shows loss minimization in action



Why Simple Updates Are Not Enough

■ Problems with Basic Gradient Descent

- Learning can be very slow
- Oscillates in zig-zag paths
- Gets stuck in local minima
- Same learning rate for all weights
- Sensitive to noisy data

■ Real training is **not smooth and simple**

Intuition Before Momentum

- **Imagine Pushing a Ball**
 - Flat surface → slow movement
 - Small pit → ball stops
 - Direction keeps changing → unstable motion
- We need **memory of past movement**

Momentum Optimizer

■ Core Idea

- Uses past gradients to build speed
- Moves consistently in the right direction

■ Simple Explanation

- Like a rolling ball gaining speed
- Doesn't stop in small pits

■ What Momentum Fixes

- Faster learning
- Reduced oscillation
- Escapes small local minima

Momentum Optimizer – Mathematical Calculations

- Recall the Basic Update Rule
 - $w = w - \alpha \times \text{gradient}$
- Problem:
 - Each step depends only on **current gradient**
 - Direction may keep changing
- Momentum – What Changes?
 - Momentum adds **memory of past updates**.
 - We now keep a new variable:
 - $v \rightarrow$ velocity (accumulated movement)
 - Two-step update:
 - $v = \beta \times v + \alpha \times \text{gradient}$
 - $w = w - v$
 - β (beta) is the **momentum factor** (usually 0.9)

Numerical Example Setup

■ Assume:

- Initial weight: $w = 10$
- Learning rate: $\alpha = 0.1$
- Momentum factor: $\beta = 0.9$
- Initial velocity: $v = 0$

■ Step 1

- $gradient = 4$

■ Velocity update:

- $v = 0.9 \times 0 + 0.1 \times 4$
- $v = 0.4$

■ Weight update:

- $w = 10 - 0.4 = 9.6$

■ Same as basic update (no past memory yet)

Numerical Example Setup

■ Step 2

- Next gradient (loss reduced):

■ gradient = 3

■ Velocity update:

- $v = \beta \times v + \alpha \times \text{gradient}$
- $v = 0.9 \times 0.4 + 0.1 \times 3$
- $v = 0.36 + 0.3$
- $v = 0.66$

■ Weight update:

- $w = 9.6 - 0.66 = 8.94$

■ Bigger movement due to accumulated momentum

Numerical Example Setup

■ Step 3

- Next gradient = **1**

■ Velocity update:

- $v = \beta \times v + \alpha \times \text{gradient}$
- $v = 0.9 \times 0.66 + 0.1 \times 1$
- $v = 0.594 + 0.1$
- $v = 0.694$

■ Weight update:

- $w = 8.94 - 0.694 = 8.246$

- Even with small gradient, momentum keeps moving

Comparison with No Momentum

Step	Gradient	Weight (No Momentum)	Weight (Momentum)
Start	–	10.0	10.0
1	4	9.6	9.6
2	3	9.3	8.94
3	1	9.2	8.246

- Momentum reaches lower loss faster
- Momentum remembers direction
- Speeds up learning in same direction
- Helps cross small local minima

Momentum Optimizer: Code Examples

#Example 1: Plain Python

#Initial values

```
w = 10.0          # weight
alpha = 0.1       # learning rate
beta = 0.9        # momentum factor
v = 0.0          # velocity (momentum memory)
grads = [4, 3, 1] # example gradients over steps
print("Plain Python Momentum Updates")
for step, g in enumerate(grads, start=1):
    v = beta * v + alpha * g
    w = w - v
    print(f"Step {step}: gradient={g},
velocity={v:.4f}, weight={w:.4f}")
```

Plain Python Momentum Updates

```
Step 1: gradient=4, velocity=0.4000, weight=9.6000
Step 2: gradient=3, velocity=0.6600, weight=8.9400
Step 3: gradient=1, velocity=0.6940, weight=8.2460
```

Momentum Optimizer: Code Examples

```
# Example 2: NumPy-style (vector friendly)
import numpy as np
w = np.array([10.0])
v = np.zeros_like(w)
alpha = 0.1
beta = 0.9
grads = [np.array([4.0]), np.array([3.0]),
np.array([1.0])]
print("\nNumPy Momentum Updates")
for step, g in enumerate(grads, start=1):
    v = beta * v + alpha * g
    w = w - v
    print(f"Step {step}: weight={w}")
```

```
NumPy Momentum Updates
Step 1: weight=[9.6]
Step 2: weight=[8.94]
Step 3: weight=[8.246]
```


Momentum Optimizer: Code Examples

Example 3: PyTorch Built-in Momentum

```
import torch
# Simple parameter
theta = torch.tensor([10.0], requires_grad=True)
# Define optimizer with momentum
optimizer = torch.optim.SGD([theta], lr=0.1, momentum=0.9)
# gradients (simulating backprop output)
manual_grads = [4.0, 3.0, 1.0]
print("\nPyTorch Momentum (SGD + momentum)")
for step, g in enumerate(manual_grads, start=1):
    optimizer.zero_grad()
    theta.grad = torch.tensor([g]) # manually set gradient
    optimizer.step()
    print(f"Step {step}: theta={theta.item():.4f}")

# Momentum keeps a running 'velocity' of past gradients
# v = beta * v + lr * gradient
# w = w - v
```

```
PyTorch Momentum (SGD + momentum)
Step 1: theta=9.6000
Step 2: theta=8.9400
Step 3: theta=8.2460
```

Why Momentum Is Still Not Perfect

■ Remaining Issues

- Learning rate is still fixed
- Some weights need smaller steps
- Others need larger steps

■ We need **adaptive learning rates**

RMSProp Optimizer

- Root Mean Square Propagation
- Used to keep numbers from becoming too large
- **Core Idea**
 - Uses **recent gradient history**
 - Adjusts learning rate for each weight
- **Simple Explanation**
 - Difficult paths → small steps
 - Easy paths → bigger steps
- **What RMSProp Fixes**
 - Stabilizes learning
 - Handles noisy gradients
 - Better for complex surfaces

RMSProp Mathematical Calculations

- We keep a new variable:
 - $s \rightarrow$ *running average of squared gradients*
- **Update rules:**
 - $s = \beta \times s + (1 - \beta) \times (\text{gradient})^2$
 - $w = w - \alpha \times \text{gradient} / \sqrt{s + \epsilon}$
- $\beta \approx 0.9$
 - ϵ = small number (avoids division by zero)
- **Assume:**
 - Initial weight: $w = 10$
 - Learning rate: $\alpha = 0.1$
 - Decay rate: $\beta = 0.9$
 - Initial $s = 0$
 - ϵ ignored for simplicity

RMSProp Mathematical Calculations

■ Step 1

■ Gradient from loss:

- $\text{gradient} = 4$

■ Update squared-gradient average:

- $s = 0.9 \times 0 + 0.1 \times 4^2$

- $s = 1.6$

■ Weight update:

- $w = 10 - 0.1 \times 4 / \sqrt{1.6}$

- $w \approx 10 - 0.316$

- $w \approx 9.684$

■ Large gradient $\rightarrow$ step is **reduced**

RMSProp Mathematical Calculations

■ Step 2

■ Next gradient (smaller):

- $\text{gradient} = 2$

■ Update s:

- $s = 0.9 \times 1.6 + 0.1 \times 2^2$

- $s = 1.84$

■ Weight update:

- $w = 9.684 - 0.1 \times 2 / \sqrt{1.84}$

- $w \approx 9.537$

■ Smaller gradient $\rightarrow$ reasonable step

RMSProp Mathematical Calculations

■ Step 3

■ Next gradient:

- $\text{gradient} = 1$

■ Update s:

- $s = 0.9 \times 1.84 + 0.1 \times 1^2$

- $s = 1.756$

■ Weight update:

- $w = 9.537 - 0.1 \times 1 / \sqrt{1.756}$

- $w \approx 9.462$

■ Step size keeps adapting automatically

Comparison

Step	Gradient	GD Weight	RMSProp Weight
Start	–	10.0	10.0
1	4	9.6	9.684
2	2	9.4	9.537
3	1	9.3	9.462

RMSProp avoids very large jumps

RMSProp Code

```
import torch
# Simple parameter
theta = torch.tensor([10.0], requires_grad=True)
# Define RMSProp optimizer
optimizer = torch.optim.RMSprop([theta], lr=0.1, alpha=0.9,
eps=1e-8)
# Gradients (simulating backprop output)
manual_grads = [4.0, 2.0, 1.0]
print("\nPyTorch RMSProp")
for step, g in enumerate(manual_grads, start=1):
    optimizer.zero_grad()
    theta.grad = torch.tensor([g]) # manually set gradient
    optimizer.step()
    print(f"Step {step}: theta={theta.item():.4f}")

# -----
# Key Teaching Line
# -----
# RMSProp adapts step size using:
#  $s = \beta * s + (1 - \beta) * \text{gradient}^2$ 
#  $w = w - \text{lr} * \text{gradient} / \sqrt{s}$ 
# -----
```

Limitation of RMSProp

■ No Direction Memory

- Looks at gradient size
- Does not remember direction
- Updates depend only on current gradient

■ Result:

- Can move inconsistently
- Direction may change suddenly
- Slower convergence in some cases

Adam Optimizer

- Adam is a smart learning method that combines momentum and adaptive learning rates to reduce error efficiently.
- **Core Idea**
 - Momentum + RMSProp combined
 - Remembers direction/past mistakes (like Momentum)
 - Adjusts step size automatically (like RMSProp)
- **Simple Explanation**
 - Smart learner with memory
 - Learns fast and safely
- **Why Adam Is Popular**
 - Fast convergence
 - Stable learning
 - Works well in practice

What Adam Keeps Track Of

- Adam stores two values for each weight:
 - $\mathbf{m} \rightarrow$ first moment (mean of gradients)
 - Direction memory (Momentum)
 - $\mathbf{v} \rightarrow$ second moment (mean of squared gradients)
 - Adaptive step size (RMSProp)

■ Adam Update Rules

- $\mathbf{m} = \beta_1 \cdot \mathbf{m} + (1 - \beta_1) \cdot \text{gradient}$
- $\mathbf{v} = \beta_2 \cdot \mathbf{v} + (1 - \beta_2) \cdot \text{gradient}^2$
- $\mathbf{w} = \mathbf{w} - \alpha \cdot \mathbf{m} / \sqrt{(\mathbf{v})}$
- Bias correction ignored for simplicity

Adam Mathematical Calculations

■ Assume:

- Initial weight: $w = 10$
- Learning rate: $\alpha = 0.1$
- $\beta_1 = 0.9$ (momentum)
- $\beta_2 = 0.9$ (for simplicity)
- Initial $m = 0$, $v = 0$

■ Step 1

- gradient = 4

■ Update moments:

- $m = \beta_1 \cdot m + (1 - \beta_1) \cdot \text{gradient}$
- $m = 0.9 \times 0 + 0.1 \times 4 = 0.4$
- $v = \beta_2 \cdot v + (1 - \beta_2) \cdot \text{gradient}^2$
- $v = 0.9 \times 0 + 0.1 \times 4^2 = 1.6$

■ Weight update:

- $w = w - \alpha \cdot m / \sqrt{v}$
- $w = 10 - 0.1 \times 0.4 / \sqrt{1.6}$
- $w \approx 9.968$

■ Small, stable step

Adam Mathematical Calculations

■ Step 2

■ Next gradient:

- $\text{gradient} = 2$

■ Update moments:

- $m = 0.9 \times 0.4 + 0.1 \times 2 = 0.56$

- $v = 0.9 \times 1.6 + 0.1 \times 2^2 = 1.84$

■ Weight update:

- $w = 9.968 - 0.1 \times 0.56 / \sqrt{1.84}$

- $w \approx 9.927$

■ Direction + adaptive step

Adam Mathematical Calculations

■ Step 3

■ Next gradient:

- $\text{gradient} = 1$

■ Update moments:

- $m = 0.9 \times 0.56 + 0.1 \times 1 = 0.604$

- $v = 0.9 \times 1.84 + 0.1 \times 1^2 = 1.756$

■ Weight update:

- $w = 9.927 - 0.1 \times 0.604 / \sqrt{1.756}$

- $w \approx 9.881$

■ Smooth and consistent movement

Adam Code

```
import torch
# Parameter
theta = torch.tensor([10.0], requires_grad=True)
# Adam optimizer (default betas)
optimizer = torch.optim.Adam([theta], lr=0.1, betas=(0.9,
0.9), eps=1e-8)
# Same gradients as slides
manual_grads = [4.0, 2.0, 1.0]
print("\nPyTorch Adam Updates")
for step, g in enumerate(manual_grads, start=1):
    optimizer.zero_grad()
    theta.grad = torch.tensor([g])
    optimizer.step()
    print(f"Step {step}: theta={theta.item():.4f}")
```


Comparison

Optimizer	Memory	Adaptive Steps	Speed
Basic GD	✗	✗	Slow
Momentum	✓	✗	Faster
RMSProp	✗	✓	Stable
Adam	✓	✓	Fast & Stable

Thank You